

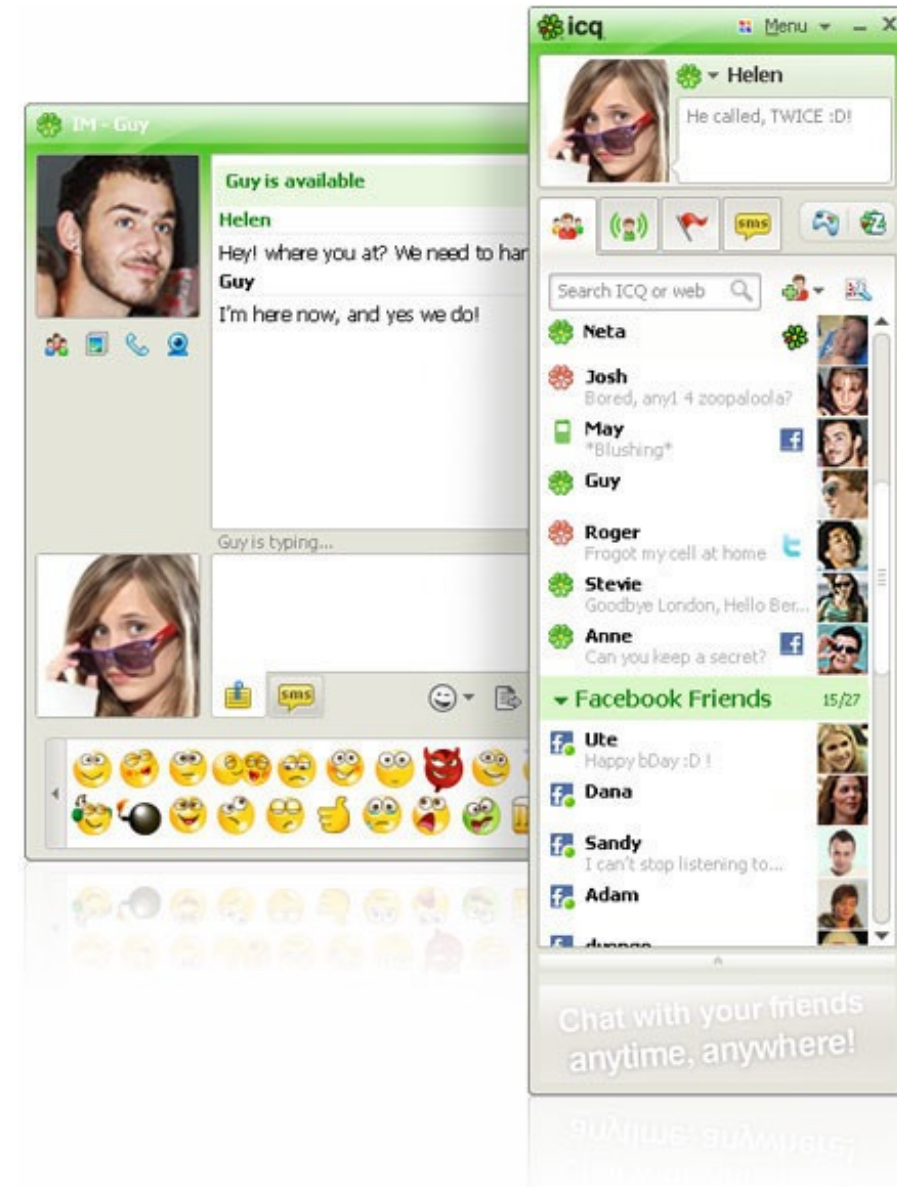
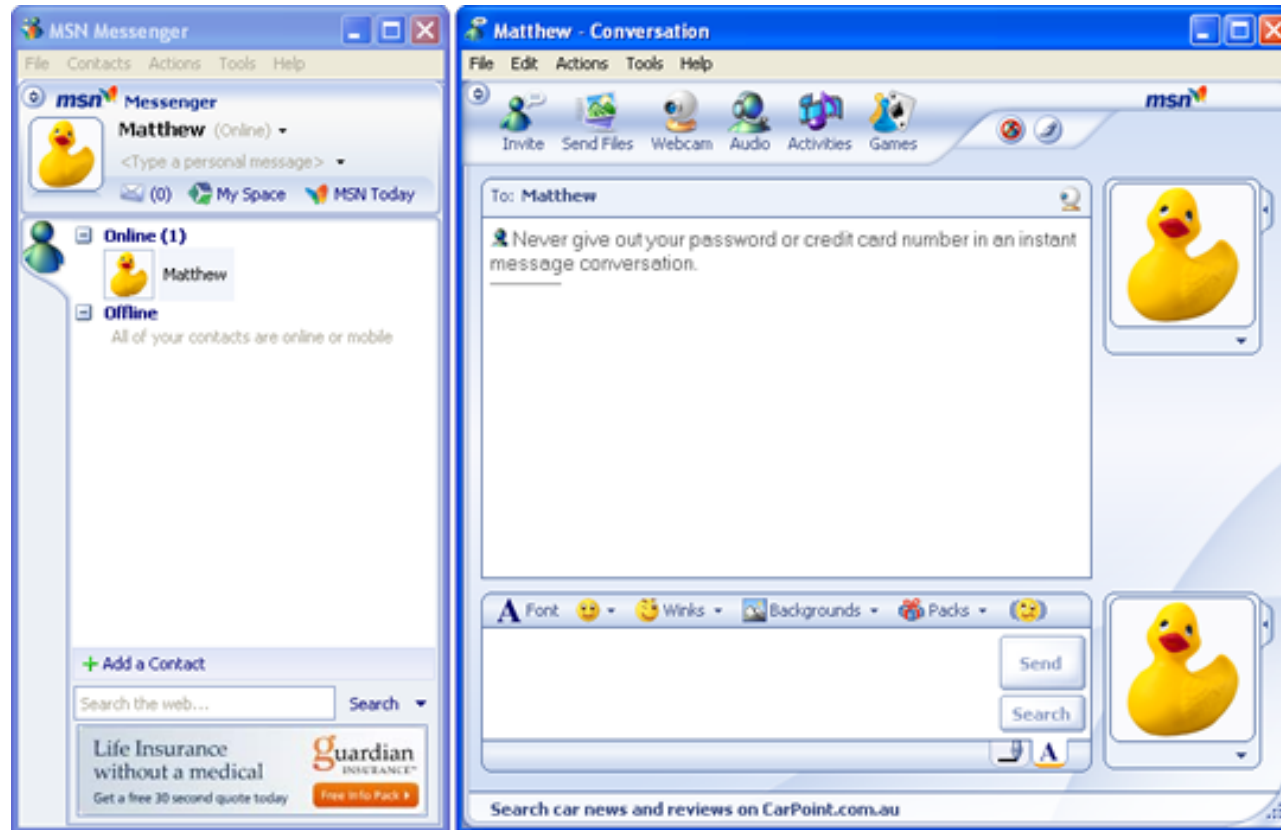
- DA343A -

Objektorienterad programutveckling, trådar och datakommunikation

Utvecklingsprojekt
(Genomgång)

Fabian Lorig

Chatt (Instant Messenger)



Beskrivning av uppgiften

- Designa och implementera ett **chatt-system** där användare kan skicka och ta emot meddelanden
- Ett meddelande kan bestå av text och/eller en bild
- Varje användare har ett användarnamn (unikt och alltid samma) och en användarbild
- Klientapplikationen består av ett grafiskt användargränssnitt
- Tre olika typer av kommunikation:
 - Uppkoppling: skicka användare
 - Meddelande: Avsändare, mottagare, text, bild, tidpunkt
 - Uppdatering: lista av uppkopplade användare

Beskrivning av uppgiften

- Servern ska klara av
 - Ett stort antal användare som ansluter sig
 - En användare avslutar anslutning till systemet
 - Uppdatera användare med en lista på alla anslutna användare (uppdatering när en användare ansluter sig / avsluter sin anslutning)
 - Meddelanden kan skickas till en eller flera mottagare
 - När mottagaren inte är uppkopplad sparas meddelandet och levereras när hen ansluter sig
 - All trafik i systemet loggas på hårddisken (samt UI för att kunna visa trafiken)
- Klienten ska kunna
 - Ansluta sig till systemet, avsluta anslutningen till systemet, skicka meddelanden, ta emot meddelanden, visa anslutna användare
 - Ha ett användarnamn och en användarbild
 - Lägga till uppkopplade användare till en kontakt-lista (sparas på hårddisken)
 - UI för att skriva text / välja bild

Krav på implementation av systemet

- TCP-kommunikation med ObjectStreams
- Servern måste hålla reda på alla anslutna klienter
- Client-instanserna och ej-skickade meddelanden ska lagras i en synkroniserad datastruktur
- Design: Boundary, control and entity-klasser

Inlämning

- Dokumentation (DA343A_UP_aa.pdf)
 - Innehåller en lista på gruppens medlemmar
 - Uttrycker vad gruppens olika medlemmar har **bidragit** med i lösningen
 - En **instruktion** så att lärare och andra grupper kan kompilera och köra applikationen
 - Dokumentation i form av **klassdiagram** och **sekvensdiagram** ritade med något verktyg
 - Alla **källkodsfiler** som ingår i projektet. Källkodsfilerna ska vara **kommenterade** så att det rimligt lätt går att förstå olika delar av koden.
- Katalogen src vilken ska innehålla alla källkodsfiler som ingår i projektet.
- Övriga filer som behövs för att köra programmet (bildfiler, ljudfiler, textfiler,...)

Formalia

- Din lösning av uppgiften lämnas in via Canvas senast **kl 14 den 10/3**.
- Inlämningen ska innehålla samtliga klasser som används i lösningen. Källkodsfilerna ska vara kommenterade så att det rimligt lätt går att förstå olika delar av koden; Javadoc krävs inte.
- Vid **redovisningen** den **14-17/3** kommer ni demonstrera ert chatt-system och presentera er kamratgranskning.
- Vi rekommenderar att utföra projektet i **grupp** (max 6 medlemmar per grupp). Ni anmäler era grupper på Canvas senast den **23/2**.
- Zip-filen ska du ge namnet **DA343A_UP_aa.zip** där aa ersätts med gruppens nummer.

Granskning

- Varje grupp ska granska en annan grupp. Senast på eftermiddagen **den 11/3** kommer filerna för kamratgranskning publiceras på Canvas.
- Din uppgift är att granska kamratens lösningar på uppgifterna avseende:
 - Kunde koden köras enligt instruktionerna i dokumentationsmaterialet?
 - Fungerar systemet som avsett? Detta innebär att ni ska kunna utföra den enligt uppgiften specificerade funktionaliteten.
 - Hur fungerar koden vid undantag? Testa att mata in felaktiga värden, trycka på knappar i fel logisk ordning och liknande.
 - Är koden bra strukturerad? Är exempelvis indelningen i klasser logisk, är metoder och variabler tydligt namngivna, med mera.
 - Innehåller dokumentationen det den ska enligt specifikationen ovan?
 - Stämmer klassdiagram och kod överens?
 - Stämmer klassdiagram och sekvensdiagram överens?
 - Stämmer interaktionen i sekvensdiagrammen överens med koden?